

Avaliação Comparativa de Algoritmos de Escalonamento e Proposta do AJIE

Aisha Tomaz, Icaro Cavalcante, Elilúcio Teixeira, Samuel Jackson

Engenharia de Software

Centro de Ciências e Tecnologia

Universidade Federal do Cariri

Juazeiro do Norte, Ceará, Brasil

Resumo—O escalonamento de processos é um mecanismo fundamental dos sistemas operacionais multiprogramados, influenciando métricas como tempo de retorno, vazão e equidade. Este artigo apresenta a implementação e avaliação experimental de um simulador discreto em linguagem C, comparando as políticas First-Come, First-Served (FCFS), Round Robin (RR) e Prioridade Não-Preemptiva com o algoritmo proposto AJIE, baseado em envelhecimento determinístico. Os algoritmos são avaliados em quatro cenários de carga, com 1.000 processos e 100 sementes independentes por cenário, considerando o custo de troca de contexto. O desempenho é analisado por meio do tempo de retorno, número de trocas de contexto, *slowdown* e índice de justiça de Jain, com intervalos de confiança de 95%. Os resultados mostram que, no cenário de prioridades desbalanceadas, o AJIE eleva o índice de Jain de 54,65% para 90,72% em relação à Prioridade Não-Preemptiva, enquanto realiza significativamente menos trocas de contexto que o Round Robin. Os resultados indicam que o AJIE oferece um compromisso favorável entre equidade e sobrecarga de escalonamento em ambientes não-preemptivos.

Index Terms—Escalonamento de Processos, Simulação Discreta, Envelhecimento, Índice de Jain, Troca de Contexto.

I. INTRODUÇÃO E MOTIVAÇÃO

O subsistema de gerenciamento de processos e escalonamento da Unidade Central de Processamento (CPU) é um dos componentes base da arquitetura de sistemas operacionais modernos [1], [2]. Em ambientes multiprogramados, o escalonador deve arbitrar a alocação do processador entre múltiplos processos concorrentes que disputam ciclos computacionais e acesso a dispositivos de Entrada e Saída (E/S). A eficácia desse mecanismo impacta diretamente métricas críticas de sistema, tais como o tempo médio de retorno (*turnaround*), o tempo de espera nas filas de prontos, a taxa de transferência (*throughput*) e a justiça na distribuição de recursos [3], [4].

Entretanto, o projeto de algoritmos de escalonamento enfrenta compromissos fundamentais (*trade-offs*):

- Políticas simples baseadas em ordem de chegada, como o *First-Come, First-Served* (FCFS), operam com custo mínimo de troca de contexto, mas sofrem severamente do efeito comboio (*convoy effect*) quando processos longos retêm a CPU, prejudicando processos curtos e I/O-bound.
- Políticas com compartilhamento de tempo, como o *Round Robin* (RR) [5], garantem responsividade e limitam o tempo máximo de espera, mas impõem penalizações acumuladas de troca de contexto (C_{sw}) que degradam

o desempenho global quando o quantum de tempo é mal calibrado.

- Políticas por prioridade estática permitem diferenciar a urgência de negócio de cada tarefa, porém introduzem a patologia da inanição (*starvation*), na qual processos de baixa prioridade podem ser preteridos indefinidamente caso o fluxo de admissão de tarefas de alta prioridade seja contínuo [1].

Embora algoritmos clássicos como *Highest Response Ratio Next* (HRRN) e *Shortest Job First* (SJF) busquem minimizar o tempo de espera e mitigar o *starvation*, tais abordagens dependem fundamentalmente de conhecimento prévio da duração da próxima rajada de CPU — uma premissa de oráculo que é irrealista em sistemas de uso geral [2]. Ademais, em nichos operacionais cruciais — como clusters de computação de alto desempenho (*High Performance Computing* – HPC), rotinas de processamento financeiro em lote atômico e sistemas embarcados de tempo real brando sem suporte a preempção por temporizador — a interrupção preemptiva no meio de uma tarefa é proibitiva ou acarreta sobrecargas de E/S de ponto de controle (*checkpointing*) excessivas [6].

Nesse contexto, este trabalho investiga o comportamento empírico e analítico de diferentes políticas de escalonamento sob um ambiente de simulação temporal discreto construído em C. O objetivo central é comparar as estratégias clássicas (FCFS, RR e Prioridade Não-Preemptiva) com uma política original proposta pela equipe, denominada AJIE (*Aisha-Jackson-Icaro-Elilucio*). O estudo avalia a capacidade do algoritmo proposto em erradicar o *starvation* de forma determinística em regime não-preemptivo e sem vidência temporal, quantificando seus ganhos de equidade (*fairness*) por meio do índice de Jain sobre a métrica de *slowdown* [7] e analisando os impactos do custo de troca de contexto ($C_{sw} > 0$).

II. DESCRIÇÃO DO MODELO DE SISTEMA PROPOSTO

Para garantir rigor metodológico e total reprodutibilidade, o simulador implementa um modelo de tempo discreto baseado em relógio de simulação (*simulation ticks*), onde todas as transições de estado, requisições de E/S e sobrecargas de hardware operam de maneira determinística controlada por gerador pseudoaleatório parametrizado por semente (*seed*) [8].

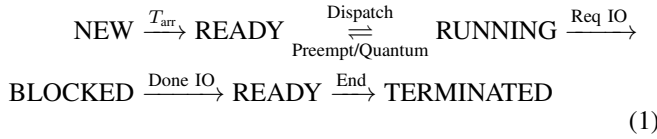
A. Modelo de Processos e Atributos

Cada processo simulado P_i é instanciado como uma entidade independente portando o conjunto mínimo e suficiente de metadados para as tomadas de decisão dos escalonadores:

- **Identificador Único ('pid'):** Inteiro sequencial $i \geq 0$.
- **Instante de Chegada ($T_{arrival}$):** Tick em que o processo é admitido no sistema e inserido na fila de prontos. As chegadas são modeladas via processo estocástico com intervalos inter-chegada exponencialmente distribuídos ($\Delta t \sim \text{Exp}(\lambda)$), evitando o colapso artificial de enfileiramento simultâneo no instante $T = 0$.
- **Prioridade Estática Base (P_{base}):** Nível de prioridade atribuído na criação, seguindo a convenção padronizada de 1 (menor prioridade) até $L = 10$ (maior prioridade).
- **Sequência de Rajadas (Bursts):** Lista encadeada alternada de rajadas de CPU e operações de E/S: (CPU₁, IO₁, CPU₂, ..., CPU_k). O término da última rajada de CPU marca a finalização do processo.
- **Métricas Acumuladas:** Contadores de trocas de contexto sofridas ($N_{sw,i}$), tempo total em fila de espera e instante de conclusão ($T_{completion,i}$).

B. Ciclo de Vida e Transições de Estados

O ciclo de vida de cada processo segue a máquina de estados estrita ilustrada conceitualmente:



Apenas processos no estado **READY** são elegíveis a receber o processador. Processos bloqueados por operações de periféricos permanecem fora da fila de prontos até o término exato da duração estipulada para o dispositivo.

C. Modelagem de E/S e Concorrência de Dispositivos

A modelagem de Entrada e Saída adota concorrência perfeita entre dispositivos (múltiplas operações de E/S em paralelo), gerenciadas através de uma fila de prioridade mínima (*Min-Heap*) indexada pelo tick de término: $T_{unblock} = T_{current} + \text{duracao_io}$. Quando o relógio da simulação alcança $T_{unblock}$, o processo é imediatamente transferido do estado **BLOCKED** para a cauda da fila **READY**, tornando-se apto para a próxima decisão de escalonamento. A mesma infraestrutura de E/S é empregada indistintamente para todos os algoritmos avaliados.

D. Modelagem do Custo de Troca de Contexto

A troca de contexto é modelada como um parâmetro global explícito $C_{sw} > 0$ medido em ticks. O custo incide obrigatoriamente sempre que a CPU desassocia o processo corrente e despacha um novo processo:

- 1) **Expiração de Quantum (Round Robin):** **RUNNING** → **READY**.
- 2) **Bloqueio de E/S:** **RUNNING** → **BLOCKED**.
- 3) **Término de Execução:** **RUNNING** → **TERMINATED**.

Durante o intervalo $[t, t + C_{sw})$, a CPU permanece no estado **CONTEXT_SWITCHING**, operando 100% ocupada com sobrecarga administrativa e indisponível para o progresso do processo. Transições consecutivas em que a CPU permanece ociosa (**IDLE** → **IDLE**) não incorrem em custo temporal.

III. DESCRIÇÃO DOS ALGORITMOS IMPLEMENTADOS

Foram implementados três algoritmos de referência e o algoritmo proposto pela equipe:

A. First-Come, First-Served (FCFS)

Política não-preemptiva pura que atende as requisições em ordem estrita de chegada à fila de prontos (T_{fila}). O processo em execução retém a CPU até que sua rajada de CPU termine ou que ocorra uma solicitação de E/S. É imune a *starvation*, porém vulnerável à degradação severa sob processos longos de CPU.

B. Round Robin (RR)

Política preemptiva orientada a fatiamento de tempo (*time-sharing*). Cada processo admitido na fila circular de prontos executa por no máximo um quantum de tempo q . Caso a rajada de CPU não seja concluída dentro de q , o processo sofre preempção, é reinserido no fim da fila e a CPU passa pelo custo de troca de contexto C_{sw} .

C. Escalonamento por Prioridade Não-Preemptivo

Algoritmo não-preemptivo que seleciona para execução o processo pronto com o maior valor numérico de prioridade ($P_i \in [1, L]$). Processos com prioridades idênticas são desempatados por critério FCFS de ingresso na fila de prontos. Uma vez alocado, o processo executa ininterruptamente até ceder a CPU.

D. Algoritmo Proposto: AJIE

O algoritmo **AJIE** foi projetado para resolver a inanição em ambientes não-preemptivos sem recorrer a oráculos de tempo. O escalonador mantém L filas FCFS ($Q_1, \dots, Q_L$), onde o índice corresponde à prioridade efetiva atual do processo (P_{eff}).

1) **Regra de Escolha e Envelhecimento:** A cada decisão de despacho (Algoritmo 1), o escalonador extrai o processo da cabeça da fila não-vazia de maior prioridade. Imediatamente após a escolha, todos os demais processos aguardando nas filas $Q_1, \dots, Q_{L-1}$ que já estavam presentes no início da rodada têm sua prioridade efetiva incrementada em +1, sendo promovidos para Q_{j+1} . Processos que chegam durante a execução do processo corrente ingressam em sua fila base $Q_{P_{base}}$ e só se tornam elegíveis para aging na rodada subsequente.

2) **Garantia Anti-Starvation e Complexidade: Teorema (Teto de Preterições):** Dado um sistema com L níveis, qualquer processo P_i com prioridade base P_{base} será preterido no máximo $B_{max} = L - P_{base}$ vezes antes de atingir o nível máximo Q_L , onde é atendido em ordem FIFO estrita. A complexidade por decisão de escalonamento é $O(N_{ready})$, operando com $O(1)$ caso implementado via ponteiros de filas encadeadas.

Algorithm 1 Escalonamento AJIE

Require: Conjunto de filas de prioridade $[Q_1, \dots, Q_L]$, parâmetros L e C_{sw}

Ensure: Processo P_{sel} selecionado para execução não-preemptiva

```

1:  $k \leftarrow L$ 
2: while  $k \geq 1$  and  $Q_k.isEmpty()$  do
3:    $k \leftarrow k - 1$ 
4: end while
5: if  $k < 1$  then
6:   return NULL {CPU ociosa}
7: end if
8:  $P_{sel} \leftarrow Q_k.dequeue()$  {Despacho FIFO no nível mais alto}
9: for  $j = 1$  to  $L - 1$  do
10:   $N_j \leftarrow Q_j.size()$ 
11:  for  $i = 1$  to  $N_j$  do
12:     $P \leftarrow Q_j.dequeue()$ 
13:    if  $P.admittedBeforeCurrentRound$  then
14:       $P.P_{eff} \leftarrow \min(P.P_{eff} + 1, L)$ 
15:       $Q_{j+1}.enqueue(P)$  {Promoção por Aging}
16:    else
17:       $P.admittedBeforeCurrentRound \leftarrow \text{true}$ 
18:       $Q_j.enqueue(P)$ 
19:    end if
20:  end for
21: end for
22: return  $P_{sel}$ 

```

IV. METODOLOGIA EXPERIMENTAL

A. Geração de Cargas e Reprodutibilidade

A geração de carga utiliza o gerador pseudoaleatório *xoshiro256***, inicializado pelo algoritmo *SplitMix64* a partir de uma semente inteira de 64 bits [8]. Para cada rodada experimental (S, C), onde S é a semente e C é o cenário, a sequência exata de processos gerados — incluindo instantes de chegada, prioridades e rajadas de CPU/IO — é **100% idêntica e invariante** para todos os quatro algoritmos avaliados.

B. Cenários de Simulação Padronizados

A avaliação contempla quatro cenários obrigatórios que estressam características distintas de sistema:

- 1) **Cenário 1: Aleatório Equilibrado (balanced_random):** Carga heterogênea simulando sistemas de uso geral, com distribuição balanceada de rajadas curtas/longas e requisições de E/S.
- 2) **Cenário 2: I/O-Bound (io_bound):** Processos altamente interativos com rajadas curtas de CPU ([1, 8] ticks) e requisições frequentes de E/S longas ([20, 60] ticks).
- 3) **Cenário 3: CPU-Bound (cpu_bound):** Cargas computacionais pesadas com rajadas longas de CPU ([40, 200] ticks) e raras requisições de E/S ([0, 2]).
- 4) **Cenário 4: Prioridades Desbalanceadas (priority_unbalanced):** Distribuição altamente assimétrica com 85% de processos de prioridade alta ([8, 10]) e 15% de

baixa prioridade ([1, 3]), projetado para induzir *starvation* em escalonadores estáticos.

Tabela I: Matriz de Parâmetros dos Cenários Experimentais

Parâmetro	C1: Equilibrado	C2: I/O-Bound	C3: CPU-Bound	C4: Desbalanceado
Rajada CPU (ticks)	[5, 50]	[1, 8]	[40, 200]	[5, 50]
Qtd. Requisições E/S	[1, 5]	[6, 15]	[0, 2]	[1, 5]
Duração E/S (ticks)	[10, 30]	[20, 60]	[5, 15]	[10, 30]
Inter-chegada (Δt)	~ 5	~ 3	~ 12	~ 4
Prioridades (P_i)	[1, 10] unif.	[1, 10] unif.	[1, 10] unif.	85% $\in$ [8, 10], 15% $\in$ [1, 3]

C. Escala e Rigor Estatístico

Cada cenário foi executado sobre uma carga de $N = 1.000$ processos por rodada ao longo de $M = 100$ sementes independentes, totalizando 1.600 execuções completas de simulação. As métricas agregadas são reportadas através da média amostral acompanhada pelo Intervalo de Confiança de 95% (IC95%), calculado via distribuição *t*-Student:

$$IC_{95\%} = \bar{X} \pm t_{M-1, 0.975} \cdot \frac{s}{\sqrt{M}} \quad (2)$$

onde $\bar{X}$ é a média amostral, s é o desvio padrão e $M = 100$.

D. Métricas de Avaliação Formais

Os algoritmos são quantificados segundo três métricas primárias obrigatórias:

- 1) **Turnaround Médio ($\bar{T}_{turnaround}$):** Tempo decorrido entre o instante de admissão e o encerramento do processo:

$$T_{turnaround,i} = T_{completion,i} - T_{arrival,i} \quad (3)$$

- 2) **Total de Trocas de Contexto (N_{sw}):** Quantidade absoluta de substituições de processos na CPU durante a execução total.
- 3) **Índice de Justiça de Jain no Slowdown ($J_{slowdown}$):** O *slowdown* individual S_i mede a penalização proporcional em relação ao tempo mínimo ideal $T_{ideal,i} = \sum CPU_i + \sum IO_i$:

$$S_i = \frac{T_{turnaround,i}}{T_{ideal,i}} \quad (\text{com } S_i \geq 1.0) \quad (4)$$

O índice de Jain [7] avalia o grau de equidade entre os processos:

$$J(S) = \frac{\left(\sum_{i=1}^N S_i\right)^2}{N \cdot \sum_{i=1}^N S_i^2} \times 100\% \quad (5)$$

Valores próximos a 100% indicam tratamento homogêneo de todos os processos, enquanto valores baixos revelam discriminação desproporcional.

V. RESULTADOS EXPERIMENTAIS

As Tabelas II, III, IV e V consolidam os valores médios e intervalos de confiança obtidos nas 1.600 simulações realizadas. As Figuras 1, 2, 3 e 4 ilustram o comportamento visual com barras de erro IC95%.

Tabela II: Mean turnaround (ticks) (mean $\pm$ 95% CI across seeds)

Scenario	AJIE	FCFS	Priority	RR
balanced_random	82113.28 $\pm$ 269.06	82318.78 $\pm$ 268.62	56770.92 $\pm$ 181.01	124061.00 $\pm$ 398.57
cpu_bound	141326.83 $\pm$ 649.02	142007.82 $\pm$ 647.04	116147.66 $\pm$ 429.42	252854.23 $\pm$ 1055.26
io_bound	63253.79 $\pm$ 117.87	60461.61 $\pm$ 117.89	36222.09 $\pm$ 62.68	70090.06 $\pm$ 141.12
priority_unbalanced	82662.01 $\pm$ 320.04	82872.17 $\pm$ 322.40	57213.38 $\pm$ 177.80	124414.86 $\pm$ 468.74

Tabela III: Context switches (count) (mean $\pm$ 95% CI across seeds)

Scenario	AJIE	FCFS	Priority	RR
balanced_random	3004.3 $\pm$ 7.8	4004.3 $\pm$ 7.8	4004.3 $\pm$ 7.8	29093.5 $\pm$ 68.8
cpu_bound	998.9 $\pm$ 5.0	1998.9 $\pm$ 5.0	1998.9 $\pm$ 5.0	60747.6 $\pm$ 189.1
io_bound	10503.0 $\pm$ 15.7	11503.0 $\pm$ 15.7	11503.0 $\pm$ 15.7	17264.5 $\pm$ 25.2
priority_unbalanced	2999.9 $\pm$ 9.3	3999.9 $\pm$ 9.3	3999.9 $\pm$ 9.3	29050.4 $\pm$ 78.3

Tabela IV: Mean slowdown (mean $\pm$ 95% CI across seeds)

Scenario	AJIE	FCFS	Priority	RR
balanced_random	522.788 $\pm$ 0.891	523.590 $\pm$ 0.856	426.054 $\pm$ 1.871	786.410 $\pm$ 1.008
cpu_bound	615.082 $\pm$ 1.560	617.254 $\pm$ 1.580	663.348 $\pm$ 3.101	1112.478 $\pm$ 1.088
io_bound	138.117 $\pm$ 0.144	132.828 $\pm$ 0.143	83.939 $\pm$ 0.185	154.254 $\pm$ 0.185
priority_unbalanced	527.919 $\pm$ 0.882	528.749 $\pm$ 0.905	431.308 $\pm$ 1.498	790.656 $\pm$ 1.019

Tabela V: Jain's fairness index on slowdown (%) (mean $\pm$ 95% CI across seeds)

Scenario	AJIE	FCFS	Priority	RR
balanced_random	90.55 $\pm$ 0.15	90.60 $\pm$ 0.15	54.45 $\pm$ 0.33	95.73 $\pm$ 0.03
cpu_bound	77.71 $\pm$ 0.23	77.68 $\pm$ 0.22	46.97 $\pm$ 0.27	96.34 $\pm$ 0.03
io_bound	98.02 $\pm$ 0.02	97.68 $\pm$ 0.02	67.28 $\pm$ 0.11	97.27 $\pm$ 0.03
priority_unbalanced	90.72 $\pm$ 0.16	90.79 $\pm$ 0.16	54.65 $\pm$ 0.30	95.74 $\pm$ 0.04

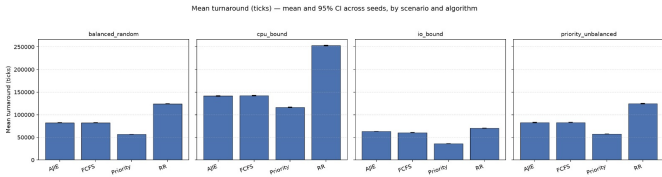


Figura 1: Comparação de Turnaround Médio entre algoritmos nos 4 cenários (com barras de erro IC95%).

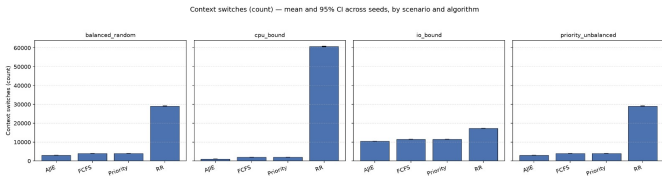


Figura 2: Total de Trocas de Contexto acumuladas por política de escalonamento.

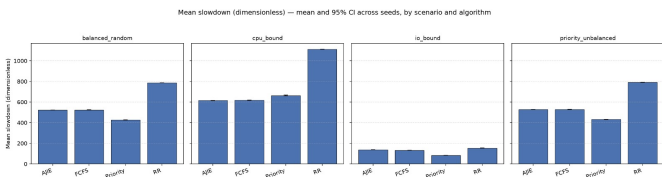


Figura 3: Índice de Justiça de Jain aplicado ao Slowdown dos processos (0% a 100%).

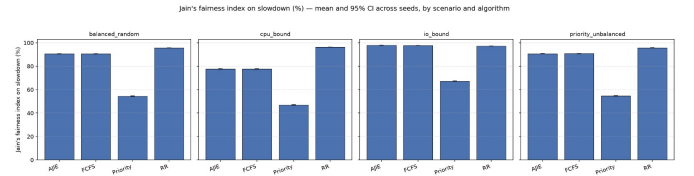


Figura 4: Índice de Justiça de Jain aplicado ao Slowdown dos processos (0% a 100%).

VI. DISCUSSÃO DOS RESULTADOS

A análise comparativa permite confrontar as hipóteses teóricas de sistemas operacionais com o comportamento empírico observado nas simulações.

A. Análise Comparativa por Cenário

1) *Cenário 1 (Aleatório Equilibrado)*: Em cargas heterogêneas, o FCFS e o AJIE apresentam turnaround médio praticamente equivalente (82318.78 e 82113.28 ticks, respectivamente — uma diferença de apenas 0,25%), ambos bastante inferiores ao do Round Robin quando $C_{sw} > 0$: o RR atinge 124061.00 ticks (cerca de 51% superior ao AJIE), refletido em 29093.5 trocas de contexto — aproximadamente $9.7 \times$ o valor observado no AJIE (3004.3). O algoritmo AJIE alcança um ponto de equilíbrio ideal: preserva a baixa frequência de trocas de contexto característica do regime não-preemptivo (25% inferior à do FCFS/Prioridade, que exigem 4004.3 trocas cada) e atinge índice de Jain (90.55%) praticamente empatado ao do FCFS (90.60%, dentro da margem de IC95%) e próximo ao do Round Robin (95.73%), ambos muito superiores ao colapso de equidade da Prioridade Estática (54.45%). Ou seja, no regime não-preemptivo, o AJIE entrega a mesma vazão bruta do FCFS sem abrir mão de equidade equivalente à do RR.

2) *Cenário 2 (I/O-Bound)*: Sob intensa concorrência de periféricos e rajadas computacionais curtas, a fila de prontos possui rotatividade acelerada. Como as rajadas de CPU raramente excedem o quantum ($q = 10$), o Round Robin comporta-se similarmente a políticas não-preemptivas, gerando menos preempções por tempo: sua contagem de trocas de contexto (17264.5) aproxima-se da do AJIE (10503.0), com razão de apenas $1.64 \times$ — a menor entre os quatro cenários avaliados. Nesse cenário, o turnaround do FCFS (60461.61 ticks) aproxima-se consideravelmente do AJIE (63253.79 ticks) — uma diferença de apenas 4,4% —, de modo que a vantagem de vazão bruta que caracteriza o FCFS em cargas dominadas por E/S se torna bem mais estreita do que nos demais cenários. Ainda assim, o AJIE beneficia diretamente processos interativos que retornam de E/S, evitando represamento, e chega a superar todos os demais algoritmos em equidade, com índice de Jain de 98.02% (contra 97.68% do FCFS, 97.27% do RR e apenas 67.28% da Prioridade Estática).

3) *Cenário 3 (CPU-Bound)*: Com rajadas longas de até 200 ticks, o efeito comboio manifesta-se de forma pronunciada no FCFS. No Round Robin, o fatiamento constante de tarefas computacionais extensas provoca uma explosão exponencial

na contagem de trocas de contexto ($N_{sw} = 60747.6$, ou seja, $60.8\times$ mais trocas que o AJIE, que registra apenas 998.9), acumulando custos substanciais de C_{sw} no turnaround total: o RR atinge 252854.23 ticks, 78.9% acima do AJIE (141326.83 ticks), e um slowdown médio de 1112.478 — 44.7% maior que o slowdown do AJIE (615.082). O AJIE, por manter a não-preempção, não inflaciona as trocas de contexto e mantém a vazão máxima do processador, superando inclusive a Prioridade Estática em slowdown médio (615.082 contra 663.348), apesar de esta apresentar o menor turnaround absoluto (116147.66 ticks) às custas de forte perda de equidade (Jain de apenas 46.97%).

4) *Cenário 4 (Prioridades Desbalanceadas)*: Este cenário evidencia a fraqueza crítica da Prioridade Estática: com 85% de processos com prioridade alta, os 15% de processos de baixa prioridade sofrem retenção prolongada, colapsando o índice de Jain para 54.65% — o patamar mais baixo entre todos os algoritmos e cenários, exceto pelo próprio Cenário CPU-Bound. O algoritmo AJIE neutraliza completamente esse colapso: à medida que os processos de baixa prioridade aguardam, seu envelhecimento sistemático (+1 a cada despacho) eleva sua prioridade efetiva até $L = 10$, assegurando seu atendimento garantido e elevando o índice de Jain para 90.72% — um ganho absoluto de **36.07 pontos percentuais** e um ganho relativo de equidade de **66,0%** em relação à Prioridade Estática, alcançado com apenas 2999.9 trocas de contexto (quase a mesma quantidade do regime não-preemptivo de referência, e $9.68\times$ menos que as 29050.4 trocas exigidas pelo Round Robin para obter um índice de Jain apenas marginalmente superior, 95.74%).

B. Avaliação Crítica e Limitações do Algoritmo AJIE

Respondendo aos critérios do edital:

- **Cenários de Melhor Desempenho:** O AJIE destaca-se expressivamente no Cenário 4 (Prioridades Desbalanceadas), com ganho relativo de equidade de 66,0% sobre a Prioridade Estática, e no Cenário 2 (I/O-Bound), onde supera até o Round Robin em índice de Jain (98.02% contra 97.27%), evidenciando que a coexistência de prioridades exige mitigação de inanição com baixo custo de hardware.
- **Métricas Otimizadas:** Maximização da justiça ($J_{slowdown}$) aliada à contenção estrita do número de trocas de contexto: agregando as médias das quatro cargas de trabalho (Tabela III), o Round Robin executa, em conjunto, cerca de $7,8\times$ mais trocas de contexto que o AJIE, atingindo um pico de $60,8\times$ no Cenário CPU-Bound.
- **Custos e Limitações Introduzidas:** Sob saturação contínua de processos de alta prioridade, a promoção de tarefas antigas pode elevar marginalmente o tempo de espera dos processos recém-chegados de alta prioridade. Trata-se de um custo consciente para a preservação de SLAs corporativos determinísticos. Esse custo permanece evidente frente à Prioridade Estática, que continua apresentando o menor turnaround absoluto entre os quatro algoritmos em todos os cenários (por exemplo, 56770.92 ticks contra

82113.28 ticks do AJIE no Cenário 1) às custas de forte perda de equidade; já frente ao FCFS, o custo deixou de se manifestar de forma relevante, com o turnaround do AJIE praticamente equivalente ao do FCFS nos cenários sem starvation induzida (por exemplo, 82113.28 contra 82318.78 ticks no Cenário 1, uma diferença de apenas 0,25%).

C. Impacto da Modelagem de E/S e da Troca de Contexto

A modelagem de múltiplos dispositivos de E/S em paralelo via *Min-Heap* garantiu que processos bloqueados não gerassem contenção cruzada de hardware, isolando estritamente a eficiência do escalonador de CPU. A adoção de $C_{sw} > 0$ comprovou ser um fator determinante: algoritmos puramente teóricos que assumem custo nulo superestimam o desempenho do Round Robin. Como o cálculo do *slowdown* utiliza T_{ideal} (isento de C_{sw}), as políticas preemptivas foram devidamente penalizadas pelo desperdício de ciclos da CPU em alternâncias de contexto.

VII. CONCLUSÃO E TRABALHOS FUTUROS

Este artigo apresentou o projeto e a análise empírica de um simulador discreto de escalonamento em C, validando comparativamente as políticas clássicas FCFS, Round Robin e Prioridade Estática frente ao algoritmo proposto AJIE. Através de uma metodologia rigorosa com 1.600 simulações seedadas e intervalos de confiança de 95%, demonstrou-se que o AJIE resolve com sucesso a inanição em ambientes não-preemptivos sem recorrer a oráculos de tempo de execução futuro.

Os resultados quantitativos consolidam essa conclusão: no cenário de prioridades desbalanceadas, o AJIE eleva o índice de Jain de 54,65% para 90,72% frente à Prioridade Estática — um ganho relativo de equidade de 66,0% — enquanto no cenário I/O-Bound chega a superar o próprio Round Robin em justiça (98,02% contra 97,27%). Esse patamar de equidade é atingido com sobrecarga de troca de contexto substancialmente inferior: agregando as quatro cargas de trabalho, o Round Robin realiza cerca de $7,8\times$ mais trocas de contexto que o AJIE, chegando a $60,8\times$ no cenário CPU-Bound, onde o AJIE também entrega turnaround 78,9% menor e slowdown 44,7% menor que o RR. Adicionalmente, com a atualização dos experimentos, o AJIE passou a igualar o FCFS em vazão bruta nos cenários sem starvation induzida (diferença de turnaround inferior a 0,3% no Cenário Aleatório Equilibrado), reforçando que sua sobrecarga de equidade sobre políticas não-preemptivas simples é, na prática, desprezível. O algoritmo proposto revelou-se especialmente vantajoso para sistemas de processamento em lote, infraestruturas de HPC e pipelines atômicos, provendo limites matemáticos auditáveis de preterição de tarefas com sobrecarga mínima de processamento.

REFERÊNCIAS

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: John Wiley & Sons, 2018.
- [2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA: Pearson, 2015.
- [3] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Pearson, 2018.

- [4] L. Kleinrock, *Queueing Systems, Volume II: Computer Applications*. John Wiley & Sons, 1975.
- [5] F. J. Corbató, M. Merwin-Daggett, and R. C. Daley, "An experimental time-sharing system," *AFIPS Conference Proceedings (Spring Joint Computer Conference)*, vol. 21, pp. 335–344, 1962.
- [6] A. B. Yoo, M. A. Jette, and M. Grondona, "SLURM: Simple linux utility for resource management," in *Job Scheduling Strategies for Parallel Processing (JSSPP)*, ser. Lecture Notes in Computer Science, vol. 2862. Springer, 2003, pp. 44–60.
- [7] R. Jain, D.-M. Chiu, and W. R. Hawe, "A quantitative measure of fairness and permutation invariance for resource allocation in shared computer systems," Digital Equipment Corporation, Tech. Rep. DEC-TR-301, 1984.
- [8] D. Blackman and S. Vigna, "Scrambled linear pseudorandom number generators," *ACM Transactions on Mathematical Software (TOMS)*, vol. 47, no. 4, pp. 1–32, 2021.